Reproducing the Temeraire Redis Results on WSL2 and Bare-Metal Linux

Aldo Costa
aldo.costa@hhu.de
Heinrich-Heine-Universität
Düsseldorf, Nordrhein-Westfalen, Deutschland

Abstract

Temeraire is a hugepage-aware extension of TCMalloc that aims to improve application performance by preserving hugepage coverage and reducing TLB pressure. The original OSDI 2021 paper evaluates Temeraire through application studies, a Google fleet experiment, and a production rollout. This report deliberately narrows reproduction to the part that can be approximated with public code: the Redis 6.0.9 case study.

The artifact evolved from a generic allocator benchmark into a Redis-focused reproduction by pinning Redis, selecting a historical public TCMalloc revision, building separate legacy and hugepage-aware shared libraries, and checking the loaded allocator at runtime. Docker/WSL2 was useful for reaching a working experiment, but long release-on runs showed throughput changes much larger than the effect in the paper. This prompted a second run on a dedicated bare-metal Debian node.

The native run changes the result. With periodic release disabled, the median Temeraire delta is −0.34% and the mean is −0.28%, with an approximate 95% interval of [−1.04%, +0.48%] that excludes the paper's +0.75%. With release enabled at the locally chosen rate of 16 MiB/s, the median is +0.55% against the paper's +0.44%. Further runs at 64 and 256 MiB/s give medians of −0.60% and −0.08%. The bare-metal measurements remove the extreme WSL2 swings and show that the small Redis result holds in one release configuration rather than across modes and rates. Throughout, the local metric is unnormalized request throughput where the paper normalizes for CPU, so these comparisons are proximity between differently constructed quantities.

1 Introduction

Memory allocation is often treated as a local implementation detail of C and C++ programs. Temeraire starts from a different observation: allocator decisions shape where objects land in memory, whether huge pages remain usable, and how much address-translation work the processor must perform later. Hunter et al. therefore connect allocator design to fleet efficiency as well as the direct cost of `malloc` and `free` [1].

A full reproduction of the Temeraire paper is outside the scope of this seminar artifact. The paper's main evidence comes from Google's production environment, including internal workloads, large-scale telemetry, and fleet-level rollout data. Those conditions are not available externally. This report therefore focuses on the Redis 6.0.9 case study: a public workload where the paper gives enough benchmark detail to support a local reproduction attempt.

The question here is narrower than rebuilding Google's internal allocator binary: can the Redis comparison be approximated with public code closely enough to observe the same small-effect regime?

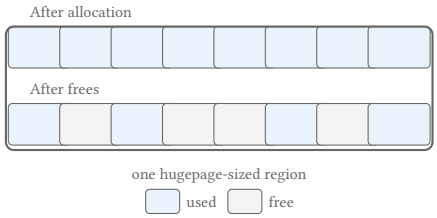

Figure 1: Fragmentation inside a hugepage-sized region. Free memory exists, but live objects prevent the whole region from being released intact. Redrawn after Figure 1 in Hunter et al. [1].

Reaching that point required two environments. Docker/WSL2 provided the development and diagnostic stage; a dedicated Linux node provided the final native measurements.

2 Background and Temeraire Mechanism

Processors translate virtual addresses to physical addresses through page tables. Because page-table walks are expensive, recent translations are cached in translation lookaside buffers (TLBs). With ordinary 4 KiB pages, large working sets can require many translations. A 2 MiB hugepage covers the same address range as 512 ordinary pages, so one hugepage TLB entry can cover much more memory than one ordinary-page entry.

The gain comes from needing fewer cached translations, not from loading 2 MiB of data into cache at once. Whether that gain is realized depends on layout: hugepage mappings require suitably aligned and contiguous regions. Linux Transparent Huge Pages (THP) can create such mappings, but its success depends partly on how the user-space allocator arranges live objects.

Figure 1 shows the central trade-off. Returning small free pieces to the OS can reduce resident memory, but it may split hugepage mappings. Keeping the hugepage intact preserves TLB coverage, but leaves some physical memory idle. Temeraire exists to make this trade-off explicit inside the allocator backend.

TCMalloc is organized as a hierarchy of caches. Many small allocations are served by upper per-CPU, transfer, or central-cache layers. Larger page-granularity requests eventually reach the pageheap, which obtains memory from the OS and releases unused spans. Temeraire modifies this backend pageheap layer rather than replacing the whole allocator.

This localization sets the shape of the reproduction. The experiment compares two backend configurations of one allocator, the legacy pageheap and the hugepage-aware path, rather than two separate allocators.

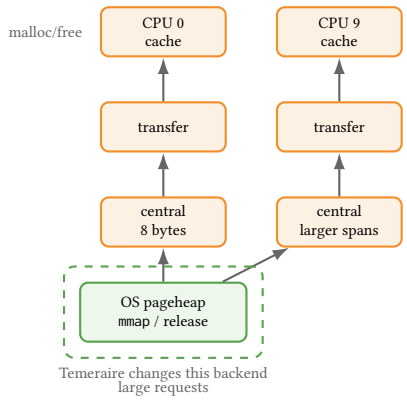

Figure 2: Simplified TCMalloc hierarchy. Temeraire changes the backend pageheap layer while leaving the upper cache structure intact. Redrawn after Figure 3 in Hunter et al. [1].

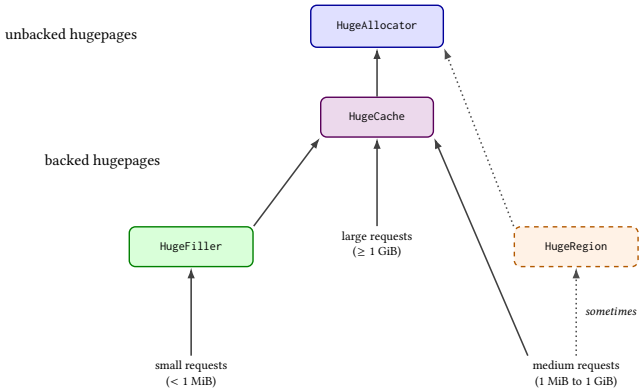

Figure 3: Temeraire's component structure. Small requests are routed to `HugeFiller`, large requests to `HugeCache`, and medium requests usually to `HugeCache` but occasionally to `HugeRegion`. Adapted after Figure 6 in Hunter et al. [1].

Figure 3 summarizes the backend components relevant for the artifact. `HugeAllocator` manages hugepage-aligned virtual address ranges. `HugeCache` keeps completely empty backed hugepages for reuse. `HugeFiller` handles small allocations and is the key component for Redis. `HugeRegion` handles medium-sized allocations that would otherwise create large slack regions.

The key Redis-relevant policy is in `HugeFiller`. For a request of size K, it prefers a hugepage whose longest free run is just large enough for the request. When several candidates are equivalent, it prefers the fuller one. This avoids consuming long free runs unnecessarily and avoids disturbing nearly empty hugepages that might otherwise drain and be released.

The Redis list workload in the paper performs small list-node and string-value allocations. These do not reach `HugeFiller` individually. As Figure 2 shows, the per-CPU and central caches serve them; only when those caches need fresh backing spans does a page-level request arrive at the pageheap. `HugeFiller` places those spans, and the legacy pageheap places them without a hugepage-aware objective. Section 4 turns that difference into the measured hypothesis.

Periodic memory release complicates the picture. When release is enabled, TCMalloc can return memory to the OS using mechanisms such as `MADV_DONTNEED`. Release interacts with hugepage coverage and adds variance, so release-off carries the cleaner placement signal of the two conditions.

3 Reproduction Target and Artifact Evolution

The original paper reports application studies, a fleet experiment, and a production rollout. Only the Redis case study is realistic for a public seminar reproduction. The paper gives Redis 6.0.9, a legacy TCMalloc pageheap baseline, 2000 `redis-benchmark` trials, 1,000,000 requests per trial, and a workload combining a five-element push with a five-element read [1]. It also states the CPU family and LLVM commit used for compilation. The exact Linux distribution, kernel version, and THP policy for the Redis run are left unstated, so this report documents them locally instead of matching them.

The repository did not initially match this target. It was a useful allocator benchmark scaffold, but its defaults used Redis 7.2.5, compared glibc against open-source gperftools TCMalloc, and did not expose a clean legacy-pageheap versus Temeraire comparison. The first reproduction step was to narrow and correct the claim: the artifact would reproduce the Redis case-study methodology only, not the paper's full evaluation.

The paper's internal allocator build is unavailable, so the artifact uses a historical public `google/tcmalloc` commit, `8e534f507074`, which still exposes the `want_no_hpaa` mechanism needed to disable the hugepage-aware path. This makes a public legacy-versus-Temeraire split possible, but it should not be read as byte-identical to Google's internal artifact.

The public repository provides no ready-made shared libraries for Redis preloading. The setup script therefore adds a small wrapper target inside the cloned TCMalloc tree and builds two loadable variants. The Temeraire build links against `//tcmalloc`; the legacy build also links against `//tcmalloc:want_no_hpaa`. The benchmark harness selects the desired library through `LD_PRELOAD`.

Bazel was helpful because it let the artifact use TCMalloc's native build graph instead of reconstructing allocator dependencies, compiler flags, and link semantics in a separate build system. It also kept the comparison narrow: both shared libraries are produced from the same wrapper source, and the intended difference is the allocator target plus the extra `//tcmalloc:want_no_hpaa` dependency in the legacy build. The cost was integration friction. Bazelisk had to be installed and pinned, the exact LLVM compiler path had to be passed through Bazel's action and repository environments, an external wrapper workspace did not inherit the right dependencies, and target visibility forced the wrapper into `tcmalloc/testing`.

Runtime verification is necessary. Redis reports `mem_allocator:libc` from its own build metadata even when `LD_PRELOAD` has replaced the process allocator, so the artifact checks `/proc/$pid/maps` to confirm that the intended shared library is mapped into the Redis process.

The final artifact reflects several failed or revised approaches. A modern `google/tcmalloc` checkout was not a convenient starting point for this wrapper strategy, and an external Bazel wrapper workspace did not naturally inherit all TCMalloc dependencies such as Abseil. The successful path was to add the wrapper inside the cloned TCMalloc source tree. Visibility was another obstacle: the `want_no_hpaa` target could not be linked from an arbitrary external package, so the wrapper had to be placed where TCMalloc's own visibility rules permitted the dependency.

A later validation run uncovered a symbol mismatch. The wrapper initially referenced background-release methods that were not present in the pinned historical TCMalloc revision. The shared objects built, but failed to load before Redis started. The fix was to resolve those methods dynamically when available, allowing the wrapper to load against the historical revision while preserving the intended behavior for newer revisions.

Compiler matching also proved more expensive than it first looked. The paper mentions LLVM commit `cd442157cf` and `-O3`. Matching that statement required a local LLVM/clang toolchain, then using it for Redis, gperftools, and the Bazel-based TCMalloc wrapper build. Later metadata confirmed use of a local LLVM 13.0.0 build from the longer `cd442157cf` commit. One older manifest still marked the exact LLVM path as disabled, so the compiler metadata is treated as authoritative.

The most important methodological difficulty was THP. Initial long runs under WSL2/Docker with THP in `madvise` mode recorded `AnonHugePages:0kB` for Redis. These runs looked like controlled allocator comparisons while leaving the hugepage mechanism itself unevidenced. The harness was then extended to record THP settings and periodic `smaps_rollup` snapshots, and the final paper-closer runs set THP to `always` before benchmarking. Even after that correction, later release-on runs showed large changes in absolute host throughput. That drift is what moved the same experiment to bare-metal Linux instead of leaving the WSL2 measurements as the final result.

4 Methodology

4.1 Common benchmark path

The repository retains a short direct Redis path for smoke tests and allocator preload checks. Measurements reported here use the paper-closer path. It fixes Redis to version 6.0.9 and uses 2000 trials, 50 clients, and pipeline depth 16. Each trial flushes the database and then runs two separate `redis-benchmark` invocations of 1,000,000 requests each:

- `LPUSHbenchmark:listv1v2v3v4v5`;
- `LRANGEbenchmark:list04`.

This structure is one reading of an underspecified sentence, and its consequences should be stated plainly. The paper specifies 2000 trials, each trial making one million requests "to push 5 elements and read those 5 elements" [1]. Read literally, one request covers both the push and the read, so a trial performs one million of each. The local trial instead issues one million pushes and then one million reads, which doubles the command count to two million. It also separates the phases: because all pushes precede all reads, `LRANGE04` returns the same final five elements on every read while the list holds five million, rather than returning each group just after that group was pushed. An interleaved implementation would keep the live set small and touch freshly written memory, which is a different access pattern and a different live-heap shape for the allocator under test. This report does not claim its choice is the paper's; it records the choice so that the comparison can be judged and repeated.

The runner stores the raw output of every operation as well as summary files, Redis logs, memory snapshots, allocator order, software revisions, and system metadata. Allocator order alternates between pairs. This does not remove drift between two consecutive blocks, but it avoids assigning the same allocator to the later position every time.

The primary metric is throughput in requests per second from `redis-benchmark`. LPUSH and LRANGE are collapsed into a combined push-plus-read rate using the harmonic mean

$$\frac{2}{1/r_{\mathrm{LPUSH}} + 1/r_{\mathrm{LRANGE}}}.$$

This treats the two throughputs as rates for sequential operations and reduces each block to one number, as the paper's single Redis delta per release mode does. The harmonic-mean aggregation itself is not specified in the paper. It is a local reconstruction choice because our public `redis-benchmark` harness records LPUSH and LRANGE as separate operation means, while the paper reports one Redis throughput delta for the combined push-five/read-five workload.

This metric is not the paper's metric, and the difference bounds every comparison in Section 5. The caption of Table 1 in Hunter et al. states that throughput there is "normalized for CPU", and the surrounding text gives the unit as requests per second per core [1]. The local figure is unnormalized: it is the request rate `redis-benchmark` reports, divided by nothing. A ratio of unnormalized rates cannot separate a genuine throughput gain from the same throughput bought with more CPU, which is exactly the distinction the paper's normalization preserves and the distinction its fleet-efficiency argument rests on. The local aggregation across two operations compounds this, since the paper reports no per-operation rates to compare against.

The consequence is stated once here and assumed afterwards. Where this report places a local value near a published one, that is numerical proximity between two differently constructed quantities. It is not agreement between two instances of one measurement, and no local number in this report can confirm or contradict the paper's normalized figures.

The hypothesis is modest, and follows from Section 2. If the placement heuristic in `HugeFiller` improves hugepage coverage for the spans backing Redis's small allocations, Redis should show a small positive throughput delta against the legacy pageheap. The effect should be clearest with periodic release disabled, since release-on adds interaction with background memory return and can mask the placement signal.

4.2 Docker/WSL2 development stage

The first complete setup used a Debian 12 container on Docker Desktop and its shared WSL2 kernel. It was where the pinned builds, preload checks, benchmark parameters, THP instrumentation, and

balanced ordering were developed. Once THP was set to `always`, Redis reported non-zero hugepage use.

The container remained useful as a reproducible development environment, but it was a poor endpoint for this particular measurement. Several hour-long release-on blocks changed in absolute throughput while the allocator was held constant. The largest apparent Temeraire improvements occurred while the host was recovering from a slower period. This made the allocator delta inseparable from the time at which each block happened.

4.3 Post-audit correction to release-on

The initial analysis treated runs labelled `paper-release-on` as periodic release experiments. A later code audit showed that this label was wrong. The runner started TCMalloc's background-actions thread but did not set its background release rate. The pinned public TCMalloc revision initializes that rate to zero. Its background loop therefore continued per-CPU-cache maintenance, but calculated zero bytes for the pageheap release call.

The existing logs remain part of the record, including a −12.02% run and its targeted rerun. They do not measure periodic pageheap release and are excluded from paper comparison. The paper does not publish its release rate, so every corrected run records the chosen rate as a local experimental parameter.

The correction itself became another part of the reproduction work. The first smoke test showed that the wrapper's dynamic symbol lookups had been failing silently, so the wrapper was changed to call the linked TCMalloc API directly. It also revealed that `LD\_PRELOAD` had leaked from the Redis server to `redis-cli` and `redis-benchmark`; the preload is now scoped to the server process. Rebuilding required one more Bazel dependency fix, followed by Linux line-ending cleanup for the container scripts. Corrected runs were started only after both allocator smoke tests logged the requested positive rate.

4.4 Bare-metal Linux stage

The WSL2 drift motivated a second workflow rather than a change to the existing container scripts. The native scripts preserve the same pinned source revisions, allocator wrappers, Redis workload, result layout, and balanced pair structure. They run directly on the dedicated node described in Table 1.

Table 1: Recorded bare-metal environment on node85.

Field	Recorded value
OS	Debian GNU/Linux 13 (trixie)
Kernel	`6.12.95+deb13-amd64`
Processor	Intel Xeon Gold 5318N
Topology	24 cores, 48 threads, one NUMA node
Memory	188 GiB
Virtualization	none detected
THP	`always`; defrag `madvise`

The move exposed a different set of practical problems. The shared home directory was too slow for Bazel's many small files, so the build and runs were moved to `/var/tmp`. The compute node could not fetch the pinned sources, which meant staging source and dependency archives for an offline build. The old LLVM revision also needed `-include cstdint` to build with Debian 13's host compiler. Finally, the German locale produced decimal commas in CSV files; the native runner now sets `LC_ALL=C`.

Three two-trial smoke runs and two ten-trial pilots came first. Four full balanced pairs then ran at 16 MiB/s, each pair covering legacy and Temeraire with release disabled and enabled. A second series kept release enabled and added four order-alternated pairs at 64 MiB/s and four at 256 MiB/s. All full allocator blocks contain 2000 trials, and each release-on Redis log confirms the requested rate. The archive holds 64,000 trial rows for the main four pairs and another 64,000 for the two added rates. A standard-library audit script re-derives every block mean from the raw trial files, checks trial identifiers, request counts, file counts, memory samples, THP state, release-rate confirmations, clean shutdowns, allocator order, and system metadata, and fails on any mismatch. Both halves of the archive pass.

5 Results

5.1 What the WSL2 runs showed

The WSL2 measurements remain useful because they explain the native rerun. With release disabled, five THP-enabled pairs produced a median of +0.48%, close to the paper's +0.75%, but individual pairs ranged from −0.76% to +3.46%. The corrected release-on sweep was less credible. Its 16 MiB/s median was −0.21% with one pair at −7.09%; the 64 MiB/s median reached +6.80%; and one 256 MiB/s pair reached +15.73%. Absolute throughput changed by more than 200 kRPS over the series and later recovered. Those swings dwarf the paper's +0.44% result. The earlier zero-rate run containing the −12.02% value stays in the protocol as a record of the discovered misconfiguration and stays out of the comparison.

The WSL2 stage therefore left an awkward picture: release-off seemed to agree with the paper, while the release-on data mainly measured when a block happened to run. Moving to node85 tested whether the small positive result would remain once the container, WSL2 kernel, and competing desktop load were removed.

5.2 Bare-metal main run

Table 2 gives the four native pairs. A positive value means that Temeraire was faster than the matched legacy run. "L first" and "T first" describe the execution order within that pair. The release-on column uses the locally chosen rate of 16 MiB/s; release-off applies no rate.

Absolute throughput on node85 was stable. All sixteen allocator blocks landed between 989 and 1003 kRPS combined, a total spread of about 1.4%, against the 200 kRPS swings recorded under WSL2. The percentage deltas therefore sit on a quiet baseline. Figure 4 shows the cost of that stability: the per-trial distributions inside each block still span roughly 5%, so the pairwise means separate distributions that overlap almost completely.

Every interval in this section is a Student-t interval computed on the log-transformed pair ratios, using the four complete pairs as the replication unit. It therefore describes the central tendency of the pair effects, not the median reported beside it; for these data the two centres differ by less than 0.01 percentage points.

The release-off result no longer supports the positive WSL2 median. Two pairs are effectively neutral or slightly positive and

Table 2: Bare-metal Temeraire throughput deltas. The release-on column uses a 16 MiB/s background release rate. The interval is a four-pair Student-t interval on the log-transformed pair ratios, so it accompanies the mean rather than the median. The paper's values use a differently constructed throughput metric.

Pair	Order	Release off	Release on
1	L first	+0.241%	+1.319%
2	T first	+0.002%	+0.149%
3	L first	−0.684%	−0.358%
4	T first	−0.686%	+0.942%
Mean		−0.282%	+0.513%
Median		−0.341%	+0.545%
95% interval (mean)		[−1.04, +0.48]	[−0.69, +1.73]
Paper		+0.75%	+0.44%

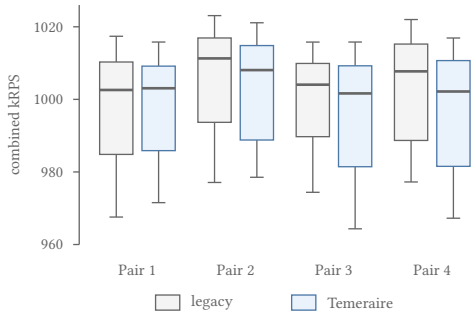

Figure 4: Per-trial combined throughput for the four release-off pairs on node85. Boxes span the interquartile range, whiskers the 5th to 95th percentile, and the heavy line the median of 2000 trials. Within-block spread is roughly 5%, an order of magnitude wider than the pairwise deltas in Table 2.

two are negative, giving a median of −0.34%, a mean of −0.28%, and an interval of [−1.04%, +0.48%]. That interval excludes the paper's +0.75%, so the native release-off aggregate is inconsistent with the paper's release-off row rather than merely quieter than it.

Release-on at 16 MiB/s points the other way. Three of four pairs favor Temeraire, with a mean of +0.51% and a median of +0.55%, both numerically near the paper's +0.44%. Three limits keep that from being a reproduction of the paper's value. The interval, [−0.69%, +1.73%], rests on four pairs and covers zero. The 2000 sequential trials inside one allocator block are autocorrelated and count as one replication rather than 2000. And the two numbers being compared are built differently, as Section 4 sets out.

5.3 Bare-metal release-rate sensitivity

The 16 MiB/s release-on column in Table 2 uses a rate that the paper never specified. Table 3 checks whether its positive result survives at two higher rates. Its first row repeats the release-on column of Table 2 for comparison; the other two rows are additional runs.

All four 64 MiB/s pairs favor legacy TCMalloc, and their interval, [−0.92%, −0.07%], excludes zero. It is the only condition here whose unadjusted interval does so, which is worth recording but is weak

Table 3: Bare-metal release-on sensitivity. Each cell compares Temeraire with the matched legacy block; the four pairs alternate allocator order. The 16 MiB/s row repeats Table 2.

Rate	Pair 1	Pair 2	Pair 3	Pair 4	Mean	Median
16 MiB/s	+1.319%	+0.149%	−0.358%	+0.942%	+0.513%	+0.545%
64 MiB/s	−0.631%	−0.094%	−0.674%	−0.574%	−0.494%	−0.603%
256 MiB/s	−0.028%	+0.402%	−0.133%	−1.419%	−0.294%	−0.080%

evidence on its own: it rests on four pairs, on a normal approximation, and on one condition picked out of the four tested without any correction for multiple comparisons. At 256 MiB/s three pairs are negative and one is positive; the −1.419% pair drives the negative mean, and its interval, [−1.54%, +0.96%], is uninformative. Inspection by trial quarter rules out a short collapse in that block, with Temeraire between 1.2% and 1.8% slower in each quarter. The sequence is not monotonic: performance changes sign between 16 and 64 MiB/s, then moves back toward zero at 256 MiB/s.

The dedicated node removes the double-digit changes seen under WSL2 and replaces one allocator verdict with three, each tied to a configuration. Release-off is slightly negative, release-on at 16 MiB/s lands near the paper's value, and the two higher-rate aggregates are negative. For release-on the result depends on a parameter the paper omits.

6 Discussion and Related Work

At the methodology level, the Redis experiment was reconstructed locally as far as the public artifact allowed: the allocator split had to be rebuilt from public code, preload had to be verified at runtime, the compiler path had to be documented, and THP activity had to be observed rather than assumed.

The move to bare metal narrows one question raised by WSL2. The double-digit release-on deltas did not recur on node85, where every pair stays within 1.5%, so they were environment- or time-dependent rather than a repeatable property of Temeraire. The rerun cannot go further than that: it changed CPU generation, kernel, operating system, execution environment, and time period together, so it isolates none of them as the cause. The move also weakens the earlier release-off story, turning a positive WSL2 median into a negative native median. Whatever differs between the two settings is large enough to change the sign of the reported effect, which is the more important point for anyone reading either number.

The release-on agreement at 16 MiB/s is encouraging but conditional. Since the paper omits its rate, 16 MiB/s has no claim to being the paper's setting, and the negative 64 and 256 MiB/s aggregates confine the numerical agreement to that one value. These data support a narrower claim: the public reconstruction lands in the paper's small positive range under one recorded configuration.

Two measurement gaps limit how far any of these comparisons can be pushed, and both are properties of the reconstruction rather than of the results. The paper normalizes Redis throughput for CPU and reports requests per second per core; the local figure is an unnormalized request rate joined across two operations by a harmonic mean this report chose. A local delta therefore cannot distinguish a throughput gain from an unchanged throughput that costs more

CPU, and it cannot confirm or contradict a normalized figure. Separately, the paper's one-million-request trial admits at least two implementations, and the sequential two-million-command form used here is one of them. Either gap alone would make a matching percentage weaker than it looks; together they mean the agreement at 16 MiB/s should be read as the local reconstruction landing in the same small-effect range, and nothing stronger.

Four pairs per condition also remain few for sub-one-percent effects. Allocator blocks run consecutively, so slow changes in frequency, temperature, or system activity can survive inside a pair. The recorded metadata omits CPU governor, turbo state, temperature, and per-block allocator mappings; preload was verified once before the run rather than inside every result directory.

Environmental fidelity also remains limited. The paper specifies the Redis version, benchmark shape, CPU family, LLVM commit, and optimization level, and leaves the Linux distribution, kernel, THP policy, and background release rate for Redis unstated. One specified property was not matched: the paper's Redis servers used Intel Skylake Xeon processors, while node85 carries a Xeon Gold 5318N, an Ice Lake generation part. TLB capacity and hugepage handling changed between those generations, so the hardware most relevant to a TLB-pressure result is the one piece of the paper's stated environment that this reproduction does not reproduce. Node85 does remove Docker and virtualization, and the exact LLVM commit was matched, but the public TCMalloc revision remains a historical approximation of Google's internal allocator.

Temeraire sits alongside other memory-management work on huge pages. SuperMalloc uses huge pages for very large allocations but does not make the general pageheap layout hugepage-aware [3]. MallocPool improves TLB behavior by serving objects from a contiguous preallocated region, but it is not a general-purpose pageheap replacement [2]. LLAMA uses lifetime prediction to group allocations that are likely to die together, whereas Temeraire uses online placement heuristics without profiling [5]. Kernel-level systems such as Ingens and HawkEye manage huge pages from the OS side and are complementary to allocator-side placement [4, 6].

7 Conclusion

This report covers the Redis case study alone. The original repository began as a generic allocator benchmark and had to be narrowed to that case study, moved to Redis 6.0.9, and shifted from glibc and gperftools comparisons to legacy TCMalloc versus Temeraire, with historical TCMalloc builds, wrapper libraries, preload checks, compiler metadata, THP controls, and memory instrumentation.

The WSL2 stage produced a working and instrumented experiment, but host drift made its release-on sweep unsuitable as final evidence. Repeating the same workload on dedicated bare-metal Linux removed those extreme swings and changed the interpretation. Release-off has a median of −0.34% and a mean of −0.28%, whose 95% interval excludes the paper's +0.75%. Release-on at 16 MiB/s has a median of +0.55% against the paper's +0.44%, while the medians at 64 and 256 MiB/s are −0.60% and −0.08%.

The reproduction therefore lands in the same small positive Redis range under one explicit release configuration rather than across the experiment as a whole. It does so with a locally reconstructed workload, an unnormalized throughput metric where the paper normalizes for CPU, and a processor one generation past the Skylake parts the paper names, so the numerical proximity is weaker evidence than the figures alone suggest. Its main contribution is the path from an unsuitable starting artifact, through build and release-control failures, to a native run whose raw data and settings can be audited. Google's fleet-scale claims and the production-era environment stay outside its reach.

References

[1] A.H. Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. 2021. Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 257–273. https://www.usenix.org/conference/osdi21/presentation/hunter

[2] Marcus Jägemar. 2018. Mallocpool: Improving Memory Performance Through Contiguously TLB Mapped Memory. In *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, Vol. 1. 1127–1130. doi:10.1109/ETFA.2018.8502619

[3] Bradley C. Kuszmaul. 2015. SuperMalloc: a super fast multithreaded malloc for 64-bit machines. In *Proceedings of the 2015 International Symposium on Memory Management* (Portland, OR, USA) *(ISMM '15)*. Association for Computing Machinery, New York, NY, USA, 41–55. doi:10.1145/2754169.2754178

[4] Youngjin Kwon, Hangchen Yu, Simon Peter, Christopher J. Rossbach, and Emmett Witchel. 2016. Coordinated and efficient huge page management with ingens. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation* (Savannah, GA, USA) *(OSDI'16)*. USENIX Association, USA, 705–721.

[5] Martin Maas, David G. Andersen, Michael Isard, Mohammad Mahdi Javanmard, Kathryn S. McKinley, and Colin Raffel. 2020. Learning-based Memory Allocation for C++ Server Workloads. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 541–556. doi:10.1145/3373376.3378525

[6] Ashish Panwar, Sorav Bansal, and K. Gopinath. 2019. HawkEye: Efficient Fine-grained OS Support for Huge Pages. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) *(ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 347–360. doi:10.1145/3297858.3304064